

Lecture: Mastering Vectors in Verilog

Introduction

The "Wire" vs. The "Bus"

Start by asking students to imagine a physical breadboard.

- **Scalar (1-bit):** A single jumper wire connecting point A to point B. It can only be 0 (0V/GND) or 1 (5V/3.3V).
- **Vector (Multi-bit):** A ribbon cable. It groups multiple wires together to carry complex data like numbers, addresses, or ASCII characters.

In Verilog, we call these groups **Vectors**. Without vectors, to represent an 8-bit number, you would need to declare:

```
4 wire d0, d1, d2, d3, d4, d5, d6, d7; //Tedious and error-prone
```

With vectors, it becomes:

```
4 wire [7:0] data; //Clean and scalable
```

Defining Vectors

Syntax

The standard syntax places the range *before* the signal name.

```
type [MSB_index : LSB_index] signal_name;
```

- **MSB:** Most Significant Bit (usually the left-most bit).
- **LSB:** Least Significant Bit (usually the right-most bit).

Examples

```
3 wire [7:0] data_bus;      // 8-bit wire (indices 7 down to 0)
4 reg  [31:0] program_ctr;  // 32-bit register
5 input [3:0] switch;       // 4-bit input port
```

Little

Endian vs. Big Endian

- **Descending ([7:0]):** This is the industry standard. Bit 7 is the MSB (highest value), Bit 0 is the LSB.
- **Ascending ([0:7]):** Valid in Verilog, but **highly discouraged** for beginners as it confuses math operations. Stick to [High:Low].

Number Literals

How do we assign values to these vectors?

Format: <size_in_bits>'<base><value>

Format	Meaning	Example (Stored as binary)
8'b1010	8-bit Binary	00001010 (Note the padding)
8'hFF	8-bit Hex	11111111
8'd10	8-bit Decimal	00001010
4'b11	4-bit Binary	0011

Crucial Rule: If the number you write is smaller than the defined size (e.g., writing 4'b11 into an 8-bit wire), Verilog pads the left side with zeros.

Vector Operations

Bit Select (Picking one wire)

Access a single bit using brackets [].

```
3  wire [7:0] a = 8'b11110000;
4  wire b;
5  assign b = a[7]; // b becomes 1
```

2.

Part Select (Slicing the ribbon cable)

Access a range of bits.

```
3  wire [31:0] dword;
4  wire [7:0] byte0;
5  assign byte0 = dword[7:0]; // Grabs the bottom 8 bits
```

Concatenation (The {} Operator)

This allows you to glue vectors together. This is one of the most useful operators in HDL.

```
3 wire [3:0] upper = 4'b1010;
4 wire [3:0] lower = 4'b0011;
5 wire [7:0] full;
6
7 assign full = {upper, lower}; // Result: 10100011
```

Replication

Repeating a bit or vector multiple times using {reps{value}}.

```
3 wire a = 1'b1;
4 wire [7:0] bus;
5
6 assign bus = {8{a}}; // Result: 11111111 (Replicates 'a' 8 times)
```

Common Pitfalls

- Reversed Indices:**
 - Bad:* wire [0:7] data; (Unless you specifically need Big Endian logic).
 - Good:* wire [7:0] data;
- Size Mismatches:**
 - Assigning a 4-bit value to a 3-bit wire will truncate the MSB (you lose data!).
 - Assigning a 4-bit value to an 8-bit wire pads with zeros (safe, but be aware of it).
- Confusing Logical vs Bitwise:**
 - & is bitwise AND (vector operation).
 - && is logical AND (true/false result).

Practical Demo with Icarus Verilog (*See the attached design and testbench module for this part*)

Let's build a module that takes two 4-bit inputs, combines them into an 8-bit vector, shifts them, and splits them up again.

Step 1: Create the Design (vector_demo.v)

Create a file named vector_demo (see attached .v file)

Step 2: Create the Testbench (vector_tb.v)

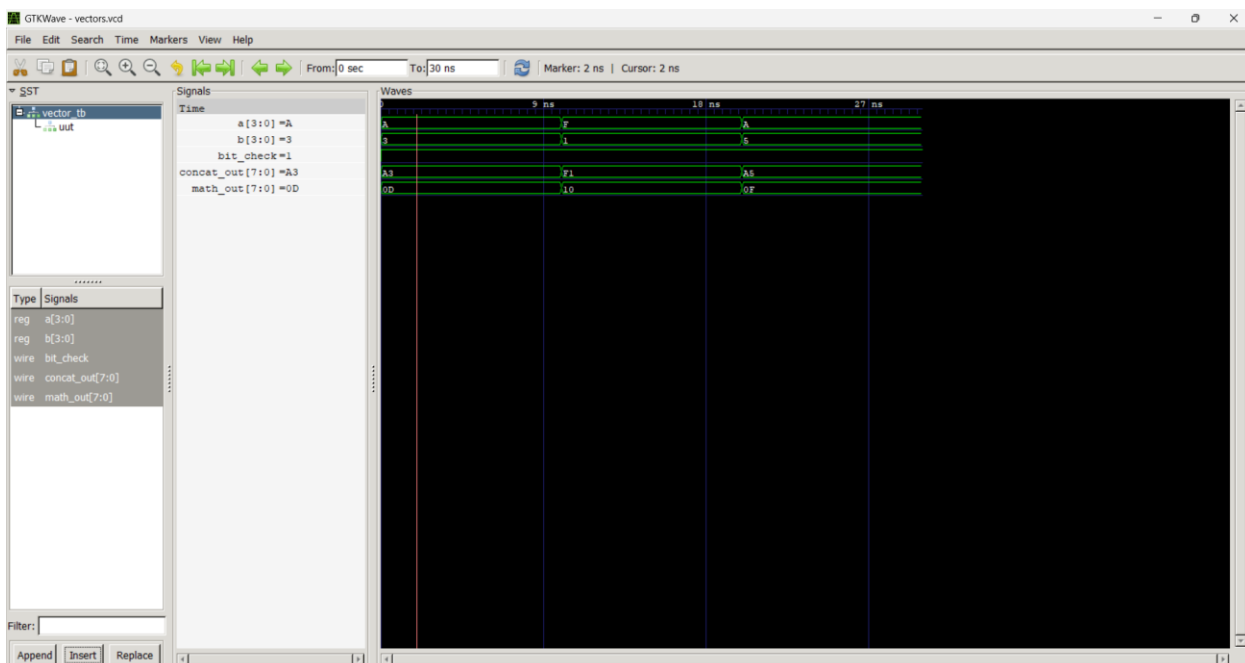
Icarus Verilog requires a testbench to simulate behavior. Create vector_demo_tb.v (see attached .v file)

Step 3: Run with Icarus Verilog and gtkwave

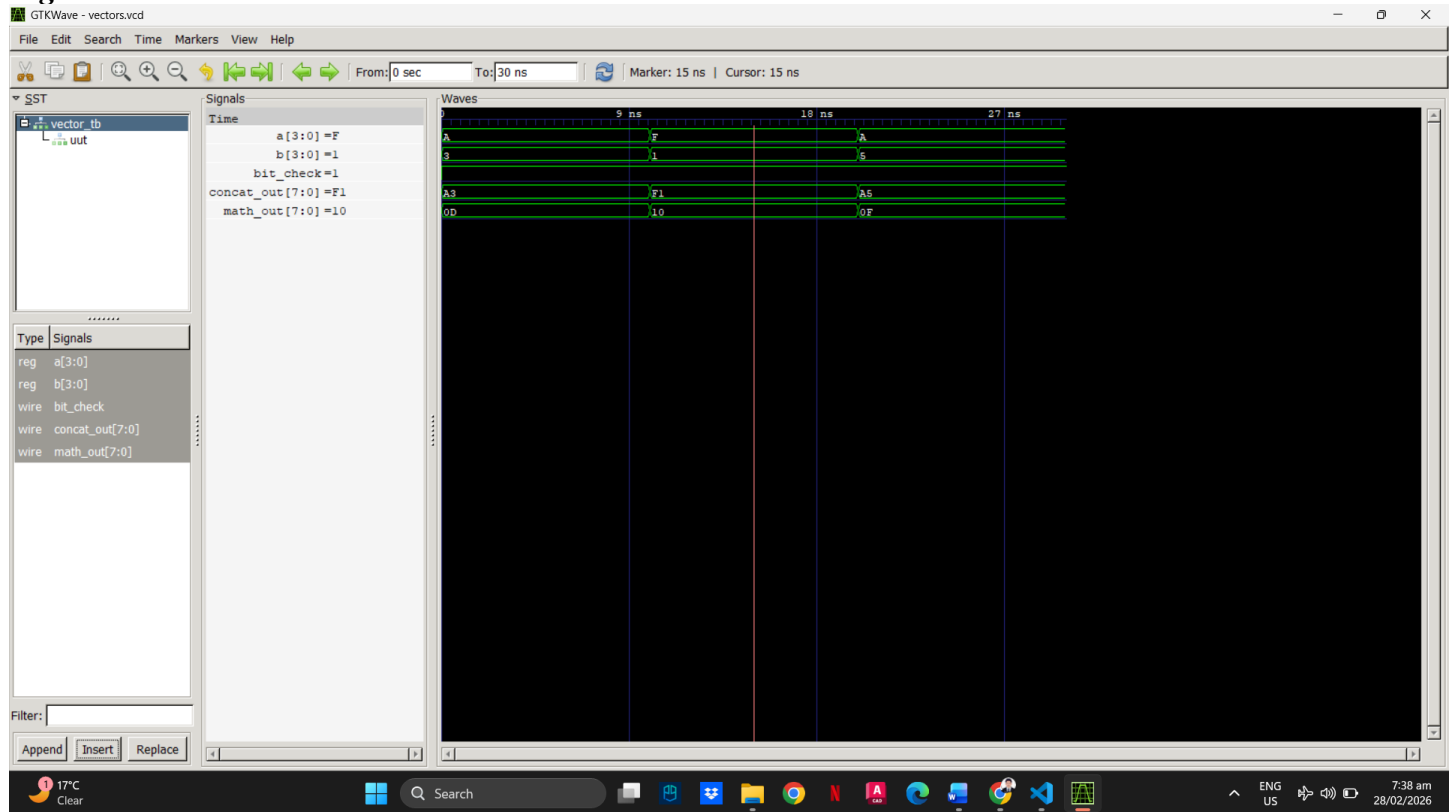
Expected Output:

```
PS C:\iverilog\bin\LSPU_Sample\Vector_Demo> iverilog -o my_vector_demo_sim vector_demo.v vector_demo_tb.v
PS C:\iverilog\bin\LSPU_Sample\Vector_Demo> vvp my_vector_demo_sim
VCD info: dumpfile vectors.vcd opened for output.
Time=0 | A=1010 B=0011 | Concat=10100011 (a3) | Sum= 13 | MSB_Check=1
Time=10 | A=1111 B=0001 | Concat=11110001 (f1) | Sum= 16 | MSB_Check=1
Time=20 | A=1010 B=0101 | Concat=10100101 (a5) | Sum= 15 | MSB_Check=1
PS C:\iverilog\bin\LSPU_Sample\Vector_Demo>
```

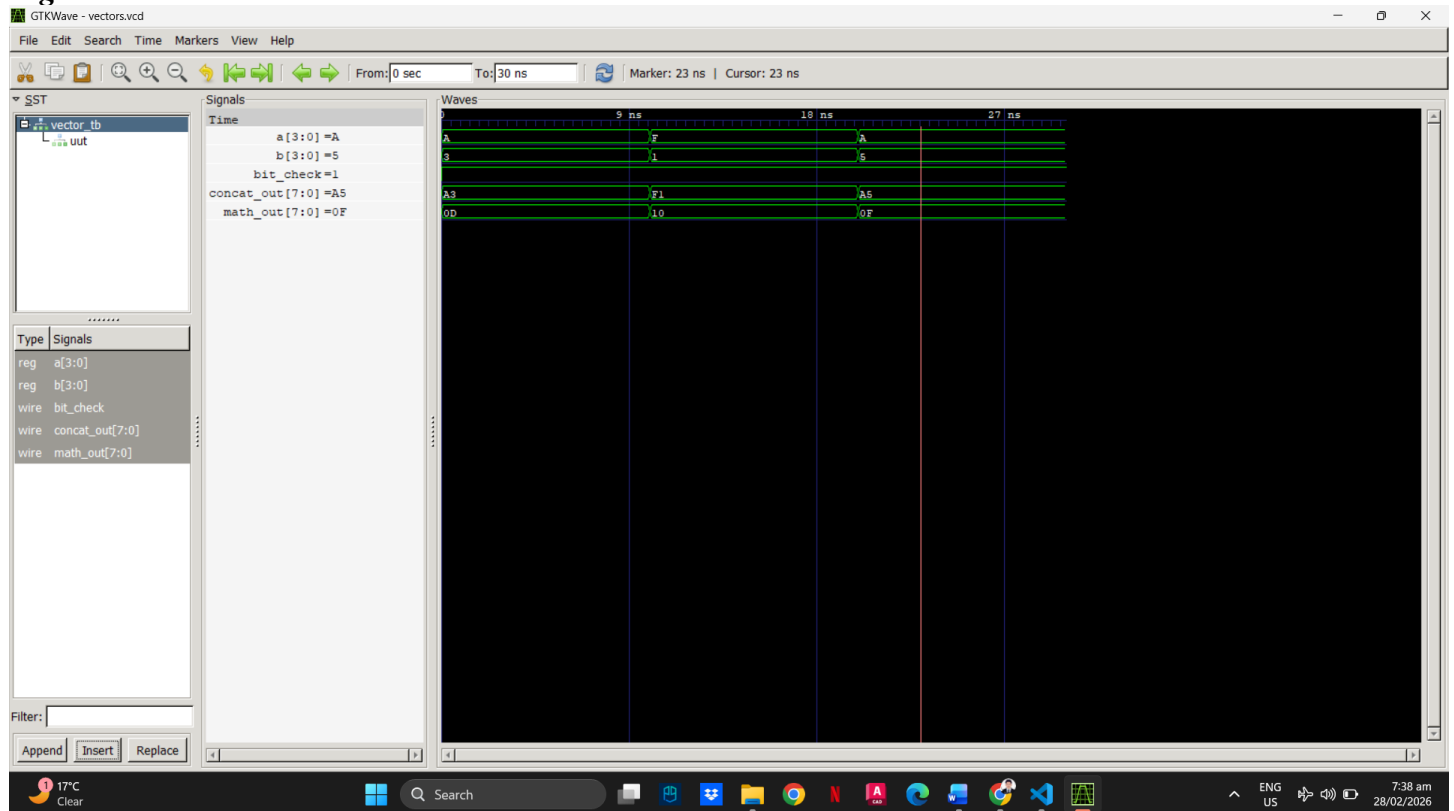
Segment 1:



Segment 2:



Segment 3:



These waveforms visualize exactly what happened inside the chip over the 30 nanoseconds of the simulation.

The General Layout

1. **Green "Boxy" Lines:** These represent **Vectors (Buses)**. Because a vector (like [3:0]) can hold many different values (0, 1, 2... 15), the line expands to show the hex value written inside.
2. **Green Single Line (bit_check):** This represents a **Scalar (1-bit)**. It is simply low (0) or high (1).
3. **The Values:** GTKWave is currently displaying values in **Hexadecimal** (Base 16). This is standard for engineering.

Segment 1: Time 0ns to 10ns

Event: The first test case in our initial begin block.

- **Inputs (a, b):**
 - a is set to **A** (which is 10 in decimal, 1010 binary).
 - b is set to **3** (0011 binary).
- **Concatenation (concat_out):**
 - The code was {a, b}.
 - Visually, it takes the A and the 3 and smashes them together.
 - Result: **concat_out [7:0] = A3** (Hex).
- **Math (math_out):**
 - Decimal logic: $10 + 3 = 13$.
 - The waveform shows **math_out[7:0] = 0D**.
 - *Why?* Because D is the hexadecimal representation of 13.
- **Bit Check:**
 - a is 1010. The MSB (left-most bit) is 1.
 - The waveform line is High (1).

Segment 2: Time 10ns to 20ns

Event: The second test case (#10).

- **Inputs (a, b):**
 - a changes to **F** (15 decimal, 1111 binary).
 - b changes to **1** (0001 binary).
- **Concatenation (concat_out):**
 - Glue F and 1 together.
 - Result: **concat_out [7:0] = F1** (Hex).
- **Math (math_out):**
 - Decimal logic: $15 + 1 = 16$
 - The waveform shows **math_out [7:0] = 10**.
 - *Warning:* Do not confuse Hex 10 with Decimal 10. In Hex, $10 = (1 \times 16) + 0 = 16$. The math is correct.
- **Bit Check:**
 - a is 1111. MSB is 1.
 - The waveform remains High.

Segment 3: Time 20ns to 30ns

Event: The third test case (Visual concatenation check).

- **Inputs (a, b):**
 - a changes to **A** (10 decimal).
 - b changes to **5** (5 decimal).
- **Concatenation (concat_out):**
 - Glue A and 5 together.
 - Result: **concat_out** [7:0] = **A5** (Hex).
- **Math (math_out):**
 - Decimal logic: $10 + 5 = 15$.
 - The waveform shows **math_out** [7:0] = **0F**.
 - F is the hexadecimal representation of 15.
- **Bit Check:**
 - a is 1010. MSB is 1.
 - The waveform remains High.

Emphasis on Concatenation vs. Addition:

1. Look at **Concat**: It literally just puts the numbers next to each other (A next to 3 becomes A3). This is how we build data packets or addresses.
2. Look at **Math**: It actually calculates the sum (A plus 3 becomes D). This is how we do processing.